

Position Generalization in Translated FashionMNIST

Architecture, patch scale, and patch embedding

Final English report

Controlled experiments · seed 42

Abstract. We test whether image classifiers retain accuracy when object position changes between training and testing. CNN is the strongest baseline and reaches 35.40% on the hardest fixed-to-random transfer. Within the ViT family, patch size 8 gives the best accuracy-cost balance. Conv2d and Flatten + Linear patch embeddings remain within 0.61 percentage points.

1 Question

A classifier may learn class appearance and object location together. This study isolates the effect of location by comparing random-position images (A) with centered images (B).

Setting	Training	Testing	Purpose
A -> A	random	random	in-distribution reference
B -> B	center	center	fixed-position reference
A -> B	random	center	random-to-fixed transfer
B -> A	center	random	hardest position shift

2 Controlled protocol

Each configuration trains one model on A and one on B. Validation chooses the checkpoint; the official test set is used once for final evaluation.

Item	Controlled choice
Data	FashionMNIST on a 64 x 64 black canvas
Split	90% train, 10% validation; official test set held out
Training	15 epochs, AdamW, seed 42
Selection	best validation checkpoint
Reporting	test accuracy and elapsed training time

Primary measure. B -> A is the hardest setting because a model trained only on centered objects must classify objects across the canvas.

Architecture and Position Shift

MLP, CNN, and patch-16 ViT under one evaluation protocol

3 Accuracy

Configuration	Parameters	A -> A	B -> B	A -> B	B -> A
MLP	542,474	71.89	89.57	71.97	13.99
CNN	205,994	92.35	93.14	92.87	35.40
ViT, patch 16	829,834	80.07	88.87	80.97	16.91

Table 1. Test accuracy (%). The best value in every column belongs to CNN.

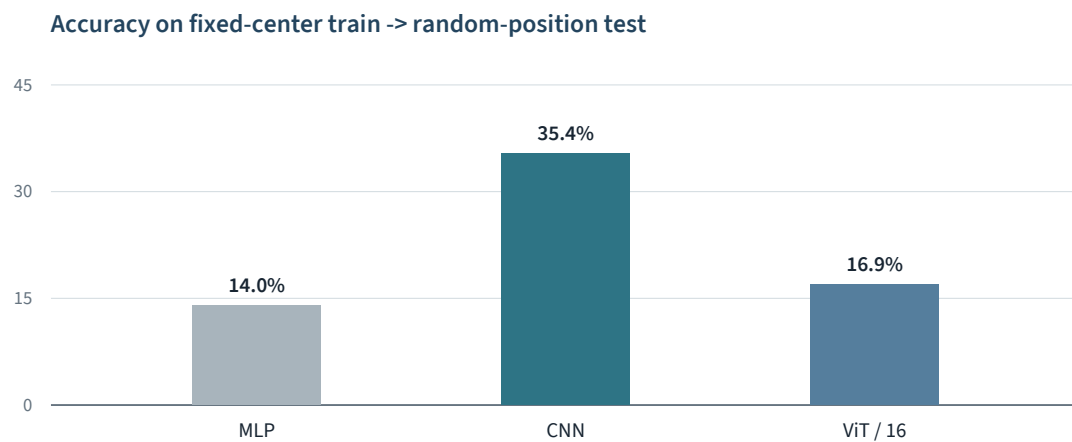


Figure 1. CNN retains substantially more accuracy on B -> A.

BEST B -> A 35.40% CNN	CNN ADVANTAGE +18.49 pp over patch-16 ViT	SMALLEST MODEL 205,994 CNN parameters
---	---	---

4 Effect of the position shift

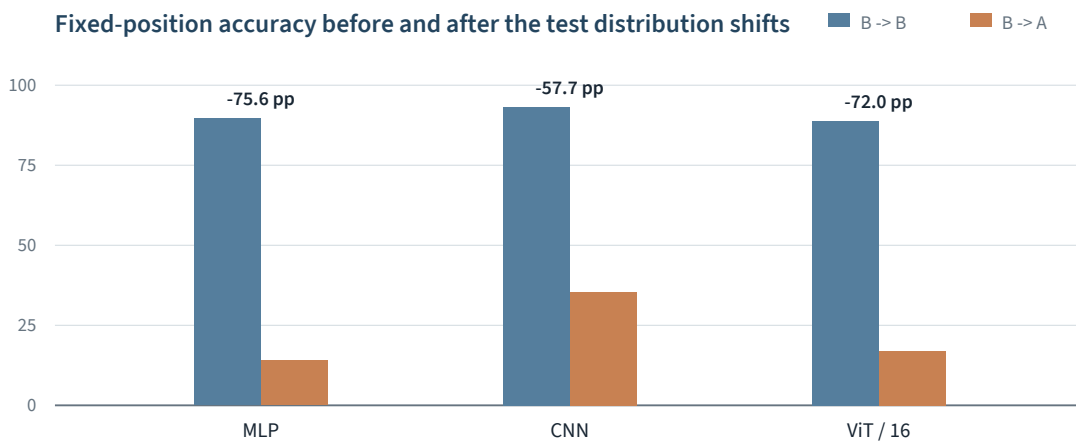


Figure 2. All models degrade after centered training is tested at random positions.

Patch Scale and Embedding

Two controlled changes to the shared Transformer configuration

5 Patch scale

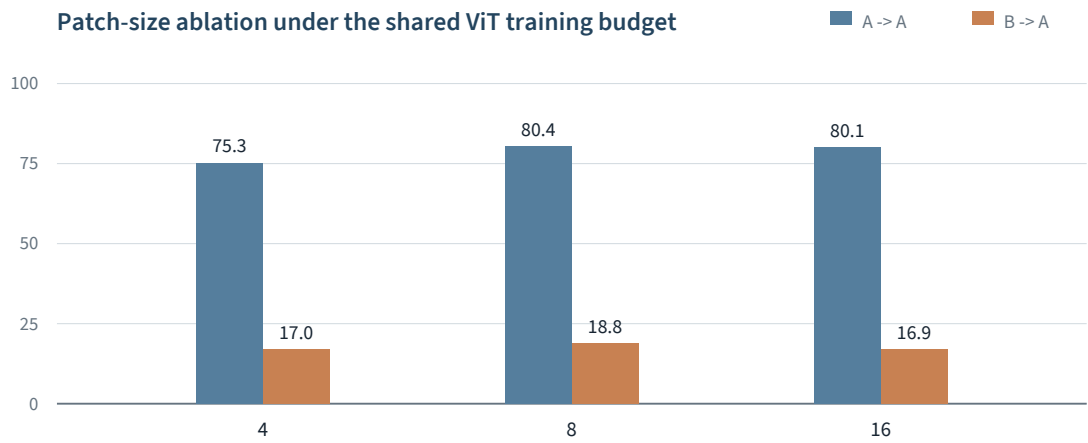


Figure 3. Patch size 8 gives the strongest observed trade-off.

Patch	Tokens	A -> A	B -> B	A -> B	B -> A	Time
4	256	75.28	87.22	74.97	17.02	8.93 min
8	64	80.40	88.27	81.13	18.81	7.36 min
16	16	80.07	88.87	80.97	16.91	6.46 min

Table 2. Accuracy (%) and total time for the A- and B-trained models.

Result. Patch size 8 leads on A -> A, A -> B, and B -> A. Patch size 4 raises the token count to 256 and is slower without an accuracy gain.

6 Patch embedding

For non-overlapping patches, a strided Conv2d and a shared linear layer perform the same type of patch-wise projection. Their results remain close.

Test	Conv2d	Flatten + Linear	Absolute gap
A -> A	80.07	79.67	0.40
B -> B	88.87	88.80	0.07
A -> B	80.97	80.48	0.49
B -> A	16.91	16.30	0.61

Table 3. Test accuracy (%) for patch size 16. Maximum absolute gap: 0.61 points.

Conclusions and Reproducibility

Concise interpretation and an exact reproduction path

7 Conclusions

01	Architecture dominates. CNN leads all four settings and retains the most accuracy under fixed-to-random transfer.
02	Patch size 8 is the practical ViT choice. It gives the best B -> A result without the 256-token cost of patch size 4.
03	Embedding syntax is not decisive. Conv2d and Flatten + Linear differ by at most 0.61 percentage points.

8 Limitations

Scope	Interpretation rule
- One formal seed; no variance estimate. - Model sizes are not exactly matched. - Runtime is hardware-specific.	Differences below 1 percentage point are treated as practically similar. Results support a controlled course comparison, not a population-level claim.

9 Reproducibility

Check	Recorded value
Code	translated_fashionmnist/experiments
Metrics	results/metrics.csv
Seed / epochs	42 / 15
Environment	Python 3.12, PyTorch 2.11, CUDA 12.8
Hardware	NVIDIA GeForce RTX 5070 Laptop GPU

```
python -m translated_fashionmnist.experiments.compare --groups all --download
```

10 Implementation map

Component	Location	Role
Dataset	data.py	construct A and B canvases
Models	models.py	MLP, CNN, ViT, patch embeddings
Protocol	experiments/protocol.py	train, validate, and evaluate
Runner	experiments/compare.py	execute all three comparison groups
Outputs	results/	metrics, history, and figures

Reproduction check. One entry point writes the metric table, run manifest, and three comparison figures to stable output paths.